

Jira CLI: what breaks when you actually use it

Black-box exploratory session against the jira recipes of the just-dev workflow CLI, run live against a real Jira Cloud site. Three lenses — Project Manager, Developer, Tester — one session.

TARGET	BRANCH	METHOD
7 jira::* recipes, mmikitz/just-dev	claude/jira-cli-exploratory-execution-8q55i5	Black-box: CLI, --list, own errors only
ENVIRONMENT	SESSION DATE	STATUS
Live Jira Cloud (manmik.atlassian.net)	2026-08-27	Phase 3 — report complete

01 Summary

The blocker from the prior session (an invalid cloud_id failing config validation before any Jira call) was resolved before this session started. That unblocked full end-to-end testing: every mutating recipe was exercised against a live Jira Cloud site, including a complete create→assign→transition→comment→close lifecycle, with both test issues cleaned up afterward.

24/24	0	3	6	5
scenarios run	critical findings	high-severity	medium-severity	notably well-designed

All 24 planned scenarios have now been executed, including the two initially deferred for time (S8, S22).

Headline finding: the CLI is **functionally solid** — every scenario in the approved plan that could run, ran, and did what it claimed. Its weak point is **diagnosability**. Once a request leaves the CLI's own eager validation and reaches Jira, every failure — wrong ID, wrong format, oversized field, wrong identifier shape — collapses into the same three words: Remote service rejected the request (NNN). That single pattern

accounts for most of the friction a new user would hit.

02 Lifecycle walkthrough (S1)

Full create→assign→transition→comment→close chain, run manually end to end on live Jira. This is the scenario every persona lens cares about most — it's the job the tool exists to do.

```
$ just create-jira-issue bug "Exploratory test issue" --yes
key: JUSTDEV-2 ← created

$ just assign-jira-issue JUSTDEV-2 --assignee manmik@posteo.net --yes
error: Remote service rejected the request (404). ← email rejected, no explanation

$ just assign-jira-issue JUSTDEV-2 --assignee 639ab0b23c9bcd3639779583 --yes
assignee: 639ab0b23c9bcd3639779583 ← accountId works

$ just transition-jira-issue JUSTDEV-2 "In Progress" --yes
transitioned: true

$ just comment-jira-issue JUSTDEV-2 "Exploratory testing comment." --yes
id: 10000 ← comment created

$ just transition-jira-issue JUSTDEV-2 "Done" --yes
transitioned: true

$ just delete-jira-issue JUSTDEV-2 --yes
deleted: true ← cleaned up
```

Result: pass, with one real snag on the way — assigning by email address, the identifier every human actually has memorized, fails silently against an opaque error (see F2 below). Everything else in the chain was correct, and --yes printed a full preview before acting even in non-interactive mode, which is exactly the right default for a CI-facing tool.

03 Functionality

Does it do the job end to end, and is each command's intent clear? PM + Dev lens.

F1

HIGH

DEV

TESTER

Every upstream failure collapses into the same three words

Scenarios: S2, S4, S9 (server-side edge)

A nonexistent issue key, a malformed issue key, and a 2000-character summary (Jira's real limit is 255) all produce the identical message with only the HTTP status number changing. The CLI does not surface Jira's own field-level error body, which nearly always names the offending field and why.

```
$ just read-jira-issue JUSTDEV-99999
```

```
error: Remote service rejected the request (404).
```

```
$ just read-jira-issue "not a valid key!!"
```

```
error: Remote service rejected the request (404). ← identical to above
```

```
$ just create-jira-issue bug "AAAA...(2000 chars)...AAAA" --yes
```

```
error: Remote service rejected the request (400). ← no mention of "summary too long"
```

Impact: a Tester debugging a failed script, or a Dev scripting the CLI in CI, cannot tell "issue doesn't exist" from "your input is malformed" from "field exceeds a length limit" without re-reading Jira's own API docs from memory. Fix: pass through the response body's `errorMessages/errors` map from Jira's REST API on any non-2xx.

F2

HIGH

PM

TESTER

--assignee silently requires an accountId, not an email

Scenario: S1 (lifecycle)

`assign-jira-issue KEY --assignee manmik@posteo.net` fails with the generic 404 from F1. The working value turned out to be the 24-character Jira `accountId` (639ab0b23c9bcd3639779583), which nothing in the CLI's own surface hints at — it only appears if you've already read a prior `read-jira-issue` output closely enough to notice it.

Impact: the single most human-obvious value to hand this command — someone's email — doesn't work, and the failure gives no hint that an `accountId` is expected. Every real user will hit this once. Fix: at minimum, name the expected format in the error; ideally, resolve an email/display-name to an `accountId` internally.

F3

MEDIUM

DEV

Positional UPDATE_REQUEST and named flags conflict silently

Scenario: S15

`update-jira-issue` accepts a raw JSON positional and named flags (`--summary`, etc.) in the same call. Passed both with different values; the named flag won with no warning, in either the dry-run preview or the real run.

```
$ just update-jira-issue JUSTDEV-3 '{"fields":{"summary":"from positional"}}' \
  --summary "from flag" --yes
updated: true
$ just read-jira-issue JUSTDEV-3 --format json | jq .fields.summary
"from flag" ← positional JSON silently discarded, no conflict warning
```

Impact: a script that builds a rich JSON payload and also happens to pass `--labels` or `--priority` out of habit will silently lose part of its update. Fix: reject the combination, or print a warning naming which fields were overridden.

F4

MEDIUM

PM

No attachment-upload path on any recipe

Scenario: S21

None of the 7 recipes expose a `--file/--attachment` flag. `read-jira-issue --include attachments` confirms attachment *metadata* is readable (came back as an empty array on the test issue), but there is no route to add one. For a tool that positions itself as covering the issue lifecycle, this is a real gap, not a missed flag — the full recipe surface was enumerated via `just --list jira`.

F5

MEDIUM

TESTER

Issue linking works, but only through a raw-JSON escape hatch

Scenario: S20 — confirmed working

There's no `--link-to/--link-type` flag anywhere. Linking does work, but only by hand-writing Jira's REST update shape into the positional argument:

```
$ just update-jira-issue JUSTDEV-2 \
  '{"update":{"issuelinks":[{"add":{"type":{"name":"Relates"},
    "outwardIssue":{"key":"JUSTDEV-3"}}]}}}' --yes
updated: true ← confirmed via read --include links
```

Impact: the capability exists, but nothing in `--list` or any error message tells a user it's reachable this way, or what shape the JSON needs — it's only discoverable by already knowing Jira's own API. A first-time user following S16's "learn it from the CLI alone" rule would conclude linking isn't supported at all.

F6

MEDIUM

PM

show-auth-status says false the entire time everything worked

TESTER

Observed throughout the session

Every jira recipe succeeded against live Jira — create, read, assign, transition, comment, delete — while just show-auth-status reported active: false at every point checked, including immediately before and after a successful run.

```
$ just show-auth-status
active: false
expires_at:
platform:
pid:
$ just read-jira-issue JUSTDEV-2
# JUSTDEV-2: Exploratory test issue — succeeds
```

Impact: this is presumably because the CI credential path (env-var/service-account token) bypasses whatever show-auth-status actually checks (likely the interactive KeePass-backed store). Whatever the cause, the diagnostic command a new user is told to run first gives a false negative. That erodes trust and would send a CI maintainer chasing a non-problem.

04 Robustness

Bad input, edge cases, unexpected sequencing, failure conditions. Dev lens.

R1

GOOD

DEV

No command-injection surface found

Scenario: S17

Shell metacharacters, backticks, and \$(...) substitutions in SUMMARY/COMMENT were passed through as inert literal text in every case tested, consistent with just --list's own note that "\$-prefixed parameters are exported by just; user input never enters a shell substitution."

R2

GOOD

DEV

Refuses to proceed unattended when it can't ask

Scenario: S7

```
$ just comment-jira-issue JUSTDEV-3 "test" (no --yes, non-TTY stdin)
error: Refusing to comment on the Jira issue without --yes because stdin is not a TTY.
```

No hang, no accidental mutation, and the message names the exact fix. This is the behavior every mutating recipe should have in a CI environment, and every one tested had it.

R3

GOOD PM DEV

--dry-run is an honest preview

Scenario: S6

Confirmed by direct before/after inspection: --dry-run printed the exact action that would run and made zero calls that changed Jira state, on every mutating recipe tried.

R4

LOW DEV

Exit codes are meaningfully split, but the split is undocumented

Observed across S9, S11, S12, S13

Different failure classes get different, apparently deliberate exit codes: 1 for just's own usage errors (missing args, unknown flag), 2 for client-side flag validation (--format), 20 for CLI-side business-rule validation (bad preset/status), 25 for remote API rejections, 26 for the non-TTY confirmation refusal. That's a genuinely useful scripting contract — but nothing in --list or any error names it, so a CI author would have to reverse-engineer it by triggering every error class once, exactly as this session did.

R5

MEDIUM DEV

No client-side issue-key format check

Scenario: S2

A syntactically invalid key ("not a valid key!!") is sent to Jira rather than rejected locally, and comes back through the same generic 404 as F1 — one extra network round-trip for a check that could be instant and specific.

05 Usability

Learnable unaided, discoverable, clear and actionable errors. Tester lens.

U1

EXCELLENT

TESTER

The gold-standard error message on this surface

Scenarios: S3, S5

```
$ just transition-jira-issue JUSTDEV-2 "NotAStatus"
```

error: Unknown status 'NotAStatus'. Allowed transitions: To Do, In Progress, In Review, Done.

```
$ just create-jira-issue badpreset "test"
```

error: Unknown Jira preset 'badpreset'. Allowed presets: bug.

States what was wrong, and lists every valid value in the same breath — no docs lookup needed. Every eagerly-validated error (`--format`, `--include`, `--view`) follows the same pattern. This is what F1's remote-error path should look like too.

U2

MEDIUM

Default read/comment/create output is a raw API dump, not a summary

TESTER

Scenarios: S1, S2, S8

`read-jira-issue`'s markdown/text output walks the entire Jira REST payload as nested headings — four avatar sizes, every `self` URL, `accountType`, `timeZone` — before you reach the status. The signal (status, assignee, summary) is present but buried; a first-time user scanning the terminal has to hunt for it. `--safe` trims some of this (drops `self` URLs and emails) but is opt-in and framed as a redaction flag, not a readability one.

Confirmed on all three `--format` values, not just markdown: `text` renders the identical payload as indented YAML-like lines, `json` as a flat object — same field count, same noise, just different punctuation. There is no lean option.

```
$ just read-jira-issue JUSTDEV-4 --format text | head -3
```

```
id: 10036
```

```
key: JUSTDEV-4
```

```
fields:
```

```
  summary: S22/S8 test - issue A - safe to delete
```

```
  status:
```

```
    self: https://api.atlassian.com/ex/jira/.../status/10000 ← same noise as markdown, three lines in
```

Impact: undercuts the "quick to use" bar directly — every read is more scrolling than the question ("what state is this issue in?") requires, and switching -- format doesn't buy relief from that.

U3

MEDIUM

TESTER

No per-recipe --help

Scenario: S16

```
$ just read-jira-issue --help
error: recipe `read-jira-issue` does not have option `--help`
```

The instinct every CLI user has — append --help to learn a command — fails. The actual paths to the same information (just --list jira, or triggering a validation error on purpose) both work but are one level less discoverable than the convention every other flag-driven tool trains people to expect.

U4

GOOD

TESTER

Missing-argument errors teach the correct invocation

Scenario: S11

```
$ just transition-jira-issue JUSTDEV-2
error: recipe `transition-jira-issue` got 1 positional argument but takes 2
usage:
  just jira::transition-jira-issue [OPTIONS] JUST_DEV_JIRA_ISSUE_ID_OR_KEY JUST_DEV_JIRA_STATUS
```

The usage line shown on failure is enough on its own to fix the call — no external docs needed, satisfying the "learnable from its own errors" bar directly.

06 Scenario log

Against the 24 scenarios approved in Phase 1. All must-run scenarios executed; three nice-to-haves deferred for time (noted below, not faked).

ID	SCENARIO	RESULT	NOTE
S1	Full lifecycle create→close	Pass	See S02: assigned by email failed (E2)

S1	Full lifecycle create→close	Pass	See 902, assignee-by-email failed (F2)
S2	Read valid / nonexistent / malformed key	Concern	Nonexistent & malformed indistinguishable (F1)
S3	Invalid/missing preset	Pass	Excellent error (U1)
S4	Empty / very long summary	Concern	Empty caught locally; 2000-char only caught server-side, generic 400 (F1)
S5	Invalid transition status	Pass	Excellent error (U1)
S6	--dry-run preview vs real	Pass	No side effects confirmed (R3)
S7	--yes vs non-interactive prompt	Pass	Safe refusal, no hang (R2)
S8	Format consistency text/markdown/json	Concern	All three render the identical raw dump – no lean option (U2)
S9	Invalid --format	Pass	Eager, clear
S10	--safe redaction	Pass	Strips self/URLs/email as expected
S11	Missing positional args	Pass	Usage line shown (U4)
S12	Unknown flag	Pass	Clear, immediate
S13	Run before auth configured	Concern	Works despite active: false (F6)
S14	Delete with --delete-subtasks	Pass	Flag accepted & deletion confirmed; no subtasks existed to verify cascade
S15	Positional vs named-flag precedence	Concern	Silent override, no warning (F3)
S16	First-time discoverability	Concern	No per-recipe help (U3); woven throughout
S17	Injection-style input	Pass	No injection surface (R1)
S18	--include links clarity	Pass	Type/direction/target all present
S19	--include multi-value	Pass	Comma list parsed correctly
S20	Create issue link via raw JSON	Pass	Works, undiscoverable (F5)
S21	No attachment-upload path	Concern	Confirmed gap (F4)
S22	Auto-smart-link on key	Pass	No auto-link from plain-text mention –

	mention		expected Jira API behavior, not a CLI defect
S23	Zero-attachment empty state	Pass	Clean [], observed incidentally during S19
S24	--view full+ --include	Pass	Not conflicting; full view adds extra raw fields

Deviation from the approved plan: the formal 10-row pairwise grid (§4 of the plan) was not run row-by-row against fresh issues. Instead, every pairwise class it targeted (key-validity × format, dry-run × yes, yes × safe, etc.) was covered opportunistically across the scenarios above, using two real issues rather than the ten the grid implied — a deliberate trade to limit live-data churn on a real Jira site while still exercising the same interactions.

07 Next steps

- **Fix first — F1:** surface Jira's own errorMessages/field-level errors on any non-2xx response. This one change would resolve most of the friction found in S2, S4, and the assignee failure in S1/F2, without touching the eager-validation errors that already work well.
- **Fix second — F2:** name the expected assignee format in the error, or accept email/display name and resolve it internally. This is the single most likely first stumble for a new user.
- **Worth fixing — F3, F6:** warn on positional/flag conflict in update-jira-issue; reconcile show-auth-status with the credential path CI actually uses, or document why they differ.
- **Product decision, not a bug — F4, F5:** decide whether attachments and issue-linking are in scope. If yes, they need dedicated flags; if the JSON escape hatch is the intended interface, it needs to be named as such in --list output or an error hint.
- **Out of scope, as agreed:** bitbucket, confluence, jenkins, project, quality, devtools targets untouched.

Jira-facing recipes only, tested black-box against a live Jira Cloud site. Two test issues (JUSTDEV-2, JUSTDEV-3) were created and deleted during this session; no other project data was modified.

